Resumen de Modelado de Procesos y LTSA

24 de abril de 2026

1. Procesos y Recursión

Un proceso se define mediante una acción seguida de un prefijo (`->`). La recursión se logra simplemente llamando al nombre del proceso al final.

```
// Un interruptor con estado
INTERRUPTOR = OFF,
OFF = (on -> ON),
ON  = (off -> OFF).
```

2. Elecciones (Choice)

El operador `|` permite al proceso elegir entre diferentes acciones.

- **Determinista:** El entorno decide basándose en etiquetas distintas. `BEBIDA = (te ->BEBIDA | cafe ->BEBIDA).`
- **No determinista:** El sistema elige internamente si hay etiquetas iguales. `MONEDA = (lanzar ->cara ->STOP | lanzar ->cruz ->STOP).`

3. Procesos y Acciones Indexados

Para evitar repetir código cuando tienes múltiples elementos similares, usas rangos [i:R].

```
const N = 3
LECTOR = (leer[i:0..N] -> LECTOR).
FILA[i:0..N] = (entrar[i] -> salir[i] -> FILA[i]).
```

4. Composición de Procesos

Para que dos o más procesos corran en paralelo, usamos el operador `||`. Los procesos se sincronizan automáticamente en las acciones que tengan el mismo nombre.

```
PROCESO_A = (sinc -> finA -> STOP).
PROCESO_B = (sinc -> finB -> STOP).
||SISTEMA = (PROCESO_A || PROCESO_B).
```

5. Renombre y Prefijos

A veces necesitas usar la misma "plantilla"de proceso pero con distintos nombres de acciones para que no choquen o para forzar una conexión.

- **Prefijo:** a:P antepone a. a todas las acciones de P. $||RED = (usuario1 : CLIENTE||usuario2 : CLIENTE)$.
- **Renombre:** nuevo/viejo cambia el nombre de una acción específica. $||CONEXION = (CLIENTE||SERVIDOR)/conectar/pedir_acceso$.

6. Compartimiento de Recursos

El patrón clásico es tener un Recurso y varios Usuarios. La sincronización se basa en que todos compartan las etiquetas de .ªdquirirz "liberar".

```
RECURSO = (get -> put -> RECURSO).
USUARIO = (get -> usar -> put -> USUARIO).
||COMPARTIDO = (u1:USUARIO || u2:USUARIO || {u1, u2}::
    RECURSO).
```

Nota sobre el operador ::

El prefijo conjunto u1, u2::RECURSO hace que el recurso esté disponible para ambos usuarios. Esto significa que la acción get del recurso se mapea a u1.get y u2.get.

7. Simulación y Bisimulación en LTS

7.1. Definición de LTS

Sea $\mathcal{S} = (S, s_0, E, \rightarrow)$ un **sistema de transiciones etiquetadas**, i.e., $\rightarrow \subseteq S \times E \times S$.

7.2. Simulación

Una relación $R \subseteq S \times S$ es una **simulación** si para todo par $(s, t) \in R$ y evento $a \in E$,

$$\forall s' : s \xrightarrow{a} s' \text{ implica } \exists t' : t \xrightarrow{a} t' \text{ y } (s', t') \in R$$

7.3. Bisimulación

Una relación $R \subseteq S \times S$ es una **bisimulación** si R y R^{-1} son ambas simulaciones. Alternativamente, para todo par $(s,t) \in R$ y evento $a \in E$:

1. $\forall s' \in S : s \xrightarrow{a} s'$ implica $\exists t' \in S : t \xrightarrow{a} t'$ y $(s',t') \in R$
2. $\forall t' \in S : t \xrightarrow{a} t'$ implica $\exists s' \in S : s \xrightarrow{a} s'$ y $(s',t') \in R$

7.4. Bisimulación débil en LTS

La transición débil $s \overset{a}{\Rightarrow} t$ se define como:

$$s \overset{a}{\Rightarrow} t \text{ es } \begin{cases} s(\xrightarrow{\tau})^* t & \text{si } a = \tau \\ s(\xrightarrow{\tau})^* \xrightarrow{a} (\xrightarrow{\tau})^* t & \text{si } a \neq \tau \end{cases}$$

8. Comunicación mediante Pasaje de Mensajes

8.1. Pasaje de Mensajes Sincrónico

Para modelar las operaciones de envío y recepción de mensajes a través de canales sincrónicos en FSP podemos usar acciones indexadas:

- `send(e, chan)` se implementa con `chan.send[e]`.
- `v = receive(chan)` se implementa con `chan.receive[v:M]`.

Listing 1: Modelo Sincrónico

```
range M = 0..9

SENDER = SENDER[0],
SENDER[e:M] = (chan.send[e] -> SENDER[(e+1)%10]).

RECEIVER = (chan.receive[v:M] -> RECEIVER).

||SyncMsg = (SENDER || RECEIVER)/{chan/chan.{send,receive
    }}.
```

8.2. Pasaje de Mensajes Asincrónico

Para modelar las operaciones de envío y recepción de mensajes a través de canales asincrónicos en FSP debemos considerar buffers acotados (debe preservarse la finitud de los modelos). Para hacer esto puede utilizarse un modelo de puertos:

Listing 2: Modelo de Puertos (Asincrónico)

```
range M = 0..9
set   S = {[M], [M][M]}

PORT              // empty state, only send permitted
    = (send[x:M] -> PORT[x]),
PORT[h:M]         // one message queued to port
    = (send[x:M] -> PORT[x][h]
      |receive[h] -> PORT
      ),
PORT[t:S][h:M]    // two or more messages queued to port
    = (send[x:M] -> PORT[x][t][h]
      |receive[h] -> PORT[t]
      ).
```

8.3. Rendezvous

El modelo `EntryDemo` ilustra la interacción entre clientes y un servidor utilizando un puerto de entrada para gestionar las peticiones.

Listing 3: Modelo de Rendezvous en FSP

```
set M = {replyA, replyB}
set S = {[M], [M][M]}

PORT              = (send[x:M] -> PORT[x]),
PORT[v:M]         = (send[x:M] -> PORT[x][v]
                    |receive[v] -> PORT),
PORT[s:S][v:M]    = (send[x:M] -> PORT[x][s][v]
                    |receive[v] -> PORT[s]).

||ENTRY = PORT/{call/send, accept/receive}.

CLIENT(CH='reply) = (entry.call[CH] -> [CH] -> CLIENT).

SERVER = (entry.accept[ch:M] -> [ch] -> SERVER).

||EntryDemo = (CLIENT('replyA) || CLIENT('replyB)
               || entry:ENTRY || SERVER ).
```

9. Lenguajes ω-regulares

Dado un lenguaje regular $A \subseteq \Sigma^*$, definimos el lenguaje de palabras infinitas A^ω como:

$$A^\omega \stackrel{\text{def}}{=} \{\sigma_1\sigma_2\sigma_3\cdots \mid \forall i \geq 0 : \sigma_i \in A \wedge \sigma_i \neq \epsilon\}$$

Es decir, A^ω contiene todas las concatenaciones infinitas de palabras no vacías de A.

Un lenguaje L se dice **ω-regular** si existen lenguajes regulares A_i y B_i $(0 \leq i \leq k)$, tales que $\epsilon \notin B_i \neq \emptyset$ y:

$$L = \bigcup_{0 \leq i \leq k} A_i \cdot B_i^\omega$$

donde $\cdot$ denota la concatenación habitual de lenguajes.

Propiedad: Los lenguajes ω-regulares son cerrados por unión, intersección y complemento.

9.1. Formalización de propiedades de Safety

Una propiedad de *safety* (seguridad) establece que "nada malo pasará".

- Si una traza viola la propiedad, lo hace en un ïnstante"finito.
- Una vez violada por un prefijo, no hay forma de remediarlo (no importa cómo continúe la traza).

Un conjunto de trazas $P \subseteq \Sigma^\omega$ es una propiedad de **safety** si cumple:

$$\forall \sigma : \sigma \notin P \implies \exists i \geq 0 : \forall \beta : \sigma[..i]\beta \notin P$$

O equivalentemente:

$$\forall \sigma : (\forall i \geq 0 : \exists \beta : \sigma[..i]\beta \in P) \implies \sigma \in P$$

donde $\sigma[..i]$ denota el prefijo i-ésimo de σ.

9.2. Formalización de propiedades de Liveness

Una propiedad de *liveness* (vitalidad) establece que .ªlgo bueno eventualmente pasará".

- Ningún fragmento de traza finito viola la propiedad por sí solo: si el evento esperado no ha ocurrido, siempre puede ocurrir más adelante.

Un conjunto de trazas $P \subseteq \Sigma^\omega$ es una propiedad de **liveness** si cumple:

$$\forall \alpha \in \Sigma^* : \exists \beta \in \Sigma^\omega : \alpha\beta \in P$$

Es decir, una propiedad P es de *liveness* si toda palabra (prefijo finito) puede ser extendida para satisfacer la propiedad. Dicho de otro modo, su complemento no contiene "prefijos malos".

9.3. Safety (Seguridad)

Definición: Establece que "nada malo sucederá jamás". Se puede violar en una traza (trace) finita. Si el sistema llega a un estado no deseado o realiza una acción prohibida, la propiedad se rompe.

- Palabra reservada property: Define un proceso "vigilante". Si el sistema intenta realizar una acción no definida en el estado actual de la property, el analizador LTSA detecta una transición al estado de ERROR (estado -1).
- Ausencia de Deadlock: Se detecta cuando el sistema llega a un estado sin transiciones salientes (excepto TERMINATED).
- Determinismo: Un proceso debe tener un único estado sucesor para cada acción; el no-determinismo suele ser un error accidental de modelado.

Listing 4: Ejemplo de propiedad de seguridad

```
property SEGURO = (pedido -> entrega -> SEGURO).
```

Si el sistema intenta hacer 'entrega' sin 'pedido', saltará un error de Safety.

9.4. Liveness (Vitalidad)

Definición: Establece que .ªlgo bueno eventualmente sucederá". Solo se viola en ejecuciones infinitas si el sistema entra en un ciclo donde la acción esperada nunca ocurre.

- Palabra reservada progress: Define acciones que deben ocurrir infinitamente a menudo.
- Livelock: Una violación donde el sistema se queda atrapado en un ciclo infinito de acciones (frecuentemente internas o irrelevantes) y nunca realiza la acción de progreso.

Listing 5: Definición de progreso

```
progress BUSCAR = {leer, escribir}
```

El analizador verificará que siempre sea posible realizar 'leer' o 'escribir'.

9.5. Prioridades (Action Priority)

Se usan para forzar comportamientos durante el análisis de las propiedades: Prioridad Alta («): El sistema elige estas acciones sobre cualquier otra si están disponibles. Útil para minimizar el tiempo entre eventos. Prioridad Baja (»): El sistema solo las elige si no hay otra opción. Útil para modelar fallos o escenarios desfavorables para encontrar violaciones de progreso.

9.6. ¿Qué le falta para ser Safety?

Para convertir P en una propiedad de safety S (hacer la clausura), seguí estos pasos: Identificá los "límites"de tus prefijos: Mirá los prefijos finitos de tu propiedad original. Buscá la traza infinita que "se queda esperando": Preguntate: "¿Hay alguna traza que cumpla con todos los prefijos pero nunca llegue al final esperado?". Agregala: Esa traza (generalmente la que se queda en el primer bucle para siempre) es la que debés sumar para cerrar el conjunto.

9.7. Descomposición S $\cap$ L

Cómo pensar la S (Safety): La S es la versión "permisiva"de tu propiedad original, donde aceptás las trazas que se quedan esperando el evento infinitamente. Lógica: "Sigue el patrón o al menos no hagas algo prohibido". Cómo pensar la L (Liveness): La L debe ser una promesa. Debe decir: "No me importa cómo empieces, pero eventualmente tenés que cumplir X". Lógica: Çualquier cosa puede pasar al principio, pero al final debe ocurrir el 'evento bueno' de la propiedad original".

10. Operadores y Modalidades

- $\Box\phi$: Siempre en el futuro sucede ϕ.
- $\Diamond\phi$: En algún instante futuro sucede ϕ.
- $\bigcirc\phi$: En el siguiente instante sucede ϕ.
- $\phi\mathbf{U}\psi$: ϕ sucede hasta que suceda ψ.
- $\phi\mathbf{W}\psi$: ϕ sucede siempre o hasta que suceda ψ (Weak Until).

11. Semántica Formal

Dada una traza σ (secuencia infinita de estados), la satisfacción de una fórmula se define inductivamente. Se denota como $\sigma[i..]$ al sufijo i-ésimo de la traza σ.

$$
\begin{aligned}
\sigma \models p &\iff p \in \sigma(0), \text{ para todo } p \in PA \\
\sigma \models \neg\phi &\iff \sigma \not\models \phi \\
\sigma \models \phi \wedge \psi &\iff \sigma \models \phi \text{ y } \sigma \models \psi \\
\sigma \models \phi \vee \psi &\iff \sigma \models \phi \text{ o } \sigma \models \psi \\
\sigma \models \bigcirc\phi &\iff \sigma[1..] \models \phi \\
\sigma \models \Diamond\phi &\iff \exists j \geq 0 : \sigma[j..] \models \phi \\
\sigma \models \Box\phi &\iff \forall j \geq 0 : \sigma[j..] \models \phi \\
\sigma \models \phi \mathbf{U} \psi &\iff \exists j \geq 0 : (\sigma[j..] \models \psi \text{ y } \forall i : 0 \leq i < j : \sigma[i..] \models \phi)
\end{aligned}
$$

12. Conceptos de Modelo y Lenguaje

Sea PA el conjunto de proposiciones atómicas.

- **Satisfacción Proposicional:** Para un conjunto $A \subseteq PA$, se cumple que $A \models p \iff p \in A$.
- **Modelo de una fórmula:** Es un subconjunto de trazas infinitas $(2^{PA})^\omega$.
- **Lenguaje de una fórmula:** $\mathcal{L}(\phi) = \{\sigma \in (2^{PA})^\omega \mid \sigma \models \phi\}$.

Nota sobre Expresividad: Los lenguajes ω-regulares son estrictamente más expresivos que LTL. Por ejemplo, el lenguaje $((\emptyset + \{p\})\{p\})^\omega$ no puede ser expresado mediante una fórmula LTL.

La relación entre los lenguajes de los sistemas, las fórmulas de lógica temporal (LTL) y los autómatas de Büchi es la base del **Model Checking**. Para entenderla detalladamente, es necesario desglosar cómo cada componente representa el comportamiento de un sistema y cómo se comparan entre sí.

12.1. 1. ¿Qué es un Autómata de Büchi?

Un autómata de Büchi ($\mathcal{A}$) es similar a un autómata finito, pero está diseñado para aceptar **trazas infinitas** (palabras de longitud infinita o ω-palabras). Se define por:

- **Alfabeto (Σ):** Un conjunto finito de símbolos.
- **Estados y Transiciones:** Un conjunto de estados con una función de transición.
- **Condición de Aceptación:** A diferencia de los autómatas finitos que aceptan si terminan en un estado final, un autómata de Büchi acepta una traza si esta pasa por un **estado de aceptación infinitas veces**.

Los autómatas de Büchi aceptan exactamente los lenguajes ω-regulares y son más expresivos que la lógica LTL.

12.2. 2. El Sistema como un Autómata

Cualquier programa o sistema con estados finitos puede representarse como una **Estructura de Kripke** o sistema de transiciones M. Este sistema genera un conjunto de ejecuciones infinitas, y el conjunto de todas estas trazas posibles es el **lenguaje del sistema**, denotado como $\mathcal{L}(M)$.

Para propósitos de verificación, este sistema M puede verse directamente como un autómata de Büchi A_P donde:

- El alfabeto son los subconjuntos de proposiciones atómicas (2^{PA}).
- **Todos los estados son de aceptación**, ya que nos interesan todas las ejecuciones posibles que el sistema pueda realizar de forma infinita.

12.3. 3. La Propiedad (LTL) como un Autómata

Cuando escribimos una fórmula LTL (ϕ), esta define un conjunto de trazas infinitas que hacen verdadera a la fórmula. Este conjunto es el **lenguaje de la propiedad**, $\mathcal{L}(\phi)$.

Un teorema fundamental establece que para toda fórmula LTL ϕ, se puede construir un autómata de Büchi A_ϕ que acepte exactamente el mismo lenguaje:

$$\mathcal{L}(A_\phi) = \mathcal{L}(\phi)$$

12.4. 4. La Relación y el Problema del Model Checking

El objetivo del Model Checking es verificar si el sistema M satisface la propiedad ϕ ($M \models \phi$). Esto ocurre si y solo si todas las trazas posibles del sistema están contenidas en el lenguaje de la propiedad:

$$\mathcal{L}(M) \subseteq \mathcal{L}(\phi)$$

Para verificar esta inclusión de forma automática, se utiliza la teoría de autómatas:

1. **Complemento de la propiedad:** En lugar de verificar la inclusión directamente, se verifica si existe alguna traza en el sistema que **no** cumpla la propiedad. Esto es equivalente a ver si la intersección entre el sistema y la negación de la propiedad es vacía:

$$\mathcal{L}(A_P) \cap \mathcal{L}(A_{\neg\phi}) = \emptyset$$

2. **Construcción del producto:** Se construye un autómata producto que representa los comportamientos comunes entre el sistema y el autómata de la propiedad negada ($A_{\neg\phi}$).

3. **Búsqueda de vaciedad:** Se utiliza un algoritmo (como un doble DFS) para buscar si existe algún ciclo que contenga un estado de aceptación en este autómata producto.
   - Si el lenguaje es **vacío**, la propiedad se cumple.
   - Si **no es vacío**, cualquier traza aceptada es un **contraejemplo**.

13. Leyes y Equivalencias

Estas leyes permiten simplificar o transformar fórmulas lógicas manteniendo su validez.

- $\neg\Box\phi \equiv \Diamond\neg\phi$
- $\neg\Diamond\phi \equiv \Box\neg\phi$
- $\neg \bigcirc \phi \equiv \bigcirc\neg\phi$
- $\Diamond\phi \equiv \text{true } \mathbf{U}\phi$
- $\Box\phi \equiv \neg\Diamond\neg\phi$
- $\Box\Box\phi \equiv \Box\phi$
- $\Diamond\Diamond\phi \equiv \Diamond\phi$
- $\phi\mathbf{U}(\phi\mathbf{U}\psi) \equiv \phi\mathbf{U}\psi$
- $(\phi\mathbf{U}\psi)\mathbf{U}\psi \equiv \phi\mathbf{U}\psi$
- $\Diamond\phi \equiv \phi \vee \bigcirc\Diamond\phi$
- $\Box\phi \equiv \phi \wedge \bigcirc\Box\phi$
- $\phi\mathbf{U}\psi \equiv \psi \vee (\phi \wedge \bigcirc(\phi\mathbf{U}\psi))$
- $(\Diamond\phi) \vee (\Diamond\psi) \equiv \Diamond(\phi \vee \psi)$
- $(\Box\phi) \wedge (\Box\psi) \equiv \Box(\phi \wedge \psi)$
- $\bigcirc(\phi\mathbf{U}\psi) \equiv (\bigcirc\phi)\mathbf{U}(\bigcirc\psi)$
- $\Diamond\Box\Diamond\phi \equiv \Box\Diamond\phi$
- $\Box\Diamond\Box\phi \equiv \Diamond\Box\phi$

14. Fairness

- Weak fairness: $\Diamond\Box\phi \to \Box\Diamond\psi \equiv \Box(\Box\phi \to \Diamond\psi)$
- Strong fairness: $\Box\Diamond\phi \to \Box\Diamond\psi \equiv \Box(\Box\Diamond\phi \to \Diamond\psi)$

El concepto de *Fairness* se usa para garantizar que, en una ejecución con múltiples procesos, todos reciban la oportunidad de progresar y no se queden "bloqueados" por decisiones arbitrarias del planificador (*scheduler*).

14.1. 1. Weak Fairness (Equidad Débil)

Se refiere a la idea de que si una acción está habilitada (lista para ejecutarse) de forma *continua e ininterrumpida*, entonces eventualmente debe ocurrir.

- *La condición:* La acción no puede "parpadear" o desactivarse; debe estar disponible todo el tiempo.
- *En LTL:* Se suele expresar como

$$\Diamond\Box(\text{habilitada}) \to \Box\Diamond(\text{ejecutada})$$

- *Ejemplo cotidiano:* Imagina que estás frente a una puerta automática. La puerta solo se abre si te quedas parado frente al sensor sin moverte. Si te retiras aunque sea un segundo, la condición se rompe y la "promesa" de que se abrirá ya no aplica bajo la equidad débil.

14.2. 2. Strong Fairness (Equidad Fuerte)

Es una condición mucho más exigente. Establece que si una acción está habilitada de forma *infinitamente frecuente* (aunque sea de forma intermitente), entonces eventualmente debe ocurrir.

- *La condición:* No importa si la acción se deshabilita temporalmente; mientras vuelva a estar lista una y otra vez, el sistema debe elegirla en algún momento.
- *En LTL:* Se expresa como

$$\Box\Diamond(\text{habilitada}) \to \Box\Diamond(\text{ejecutada})$$

- *Ejemplo cotidiano:* Imagina un semáforo. La luz verde no está encendida todo el tiempo, pero se enciende "infinitas veces" a lo largo del día. La equidad fuerte garantiza que, aunque la luz verde sea intermitente, eventualmente podrás cruzar.